Module 1 Recap

Introduction to Artificial Neural Networks

IIT Madras

Live Coding Session

From ML to Deep Learning

The key shift: features go from hand-crafted to learned

Traditional ML

Raw Data → **Feature Eng.** (manual, slow) → Model

Works on: structured/tabular data

Linear Reg, Trees, SVM, etc.

Bottleneck: feature quality = your ceiling

Struggles with images, audio, text

Deep Learning

Raw Data → **Neural Network** learns features + predicts

Works on: images, audio, text, everything

CNNs, RNNs, Transformers, etc.

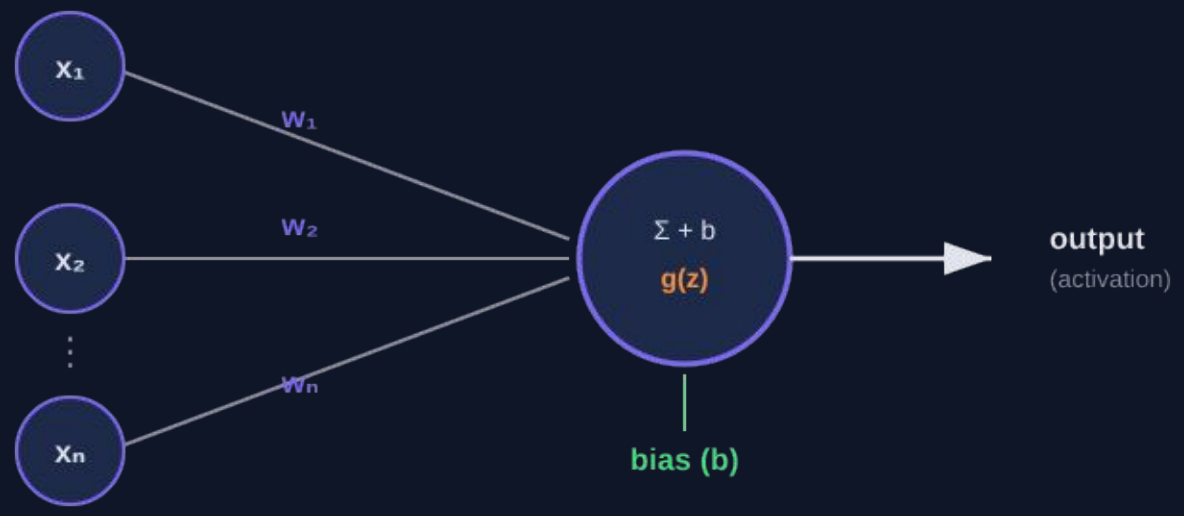
Features learned automatically

Scales with more data and compute

Key Insight: Deep Learning = representation learning. The network discovers what features matter, so you don't have to.

The Single Neuron

Every neuron = linear combination ⟶ non-linear activation



Step 1: $z = \Sigma w_i x_i + b$

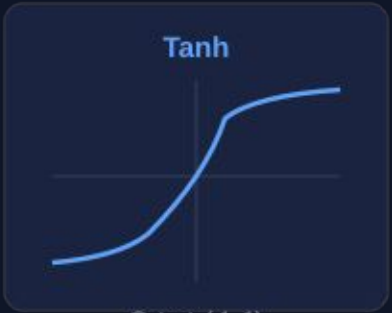
Step 2: $a = g(z)$

Activation Functions

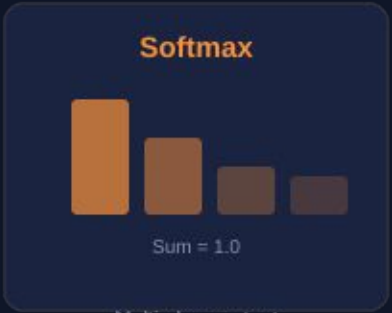
Without these, stacking layers collapses to a single linear transformation

Output: (0, 1)

Output: [0, ∞)



Output: (-1, 1)



Multi-class output

✓ Hidden Layers

Use ReLU (default). Leaky ReLU if dead neurons.

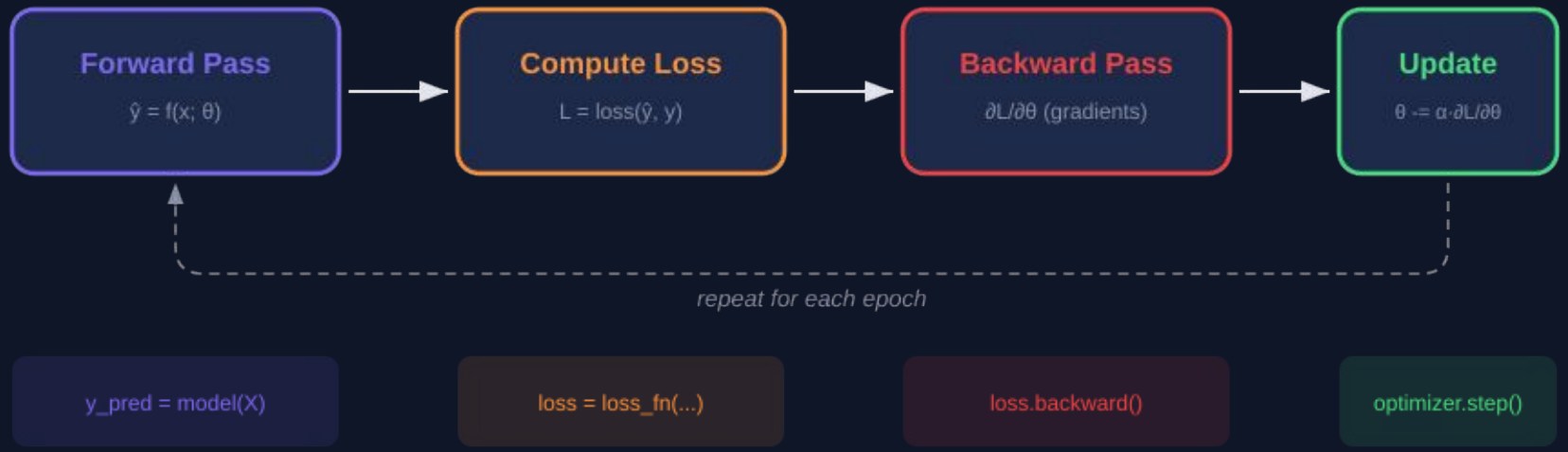
✓ Output Layer

Sigmoid → binary | Softmax → multi-class | None → regression

✗ Never use Sigmoid or Tanh in hidden layers — vanishing gradient problem kills learning in deep networks

The Learning Loop

Four steps, every epoch, every neural network ever trained

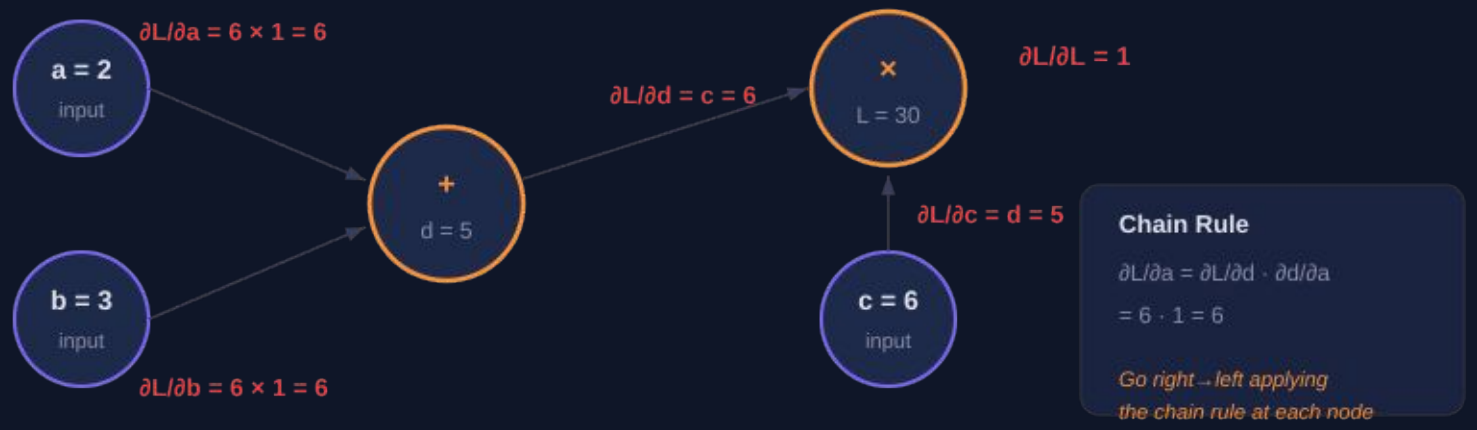


+ don't forget: optimizer.zero_grad() before backward!

PyTorch does the hard part: autograd computes all gradients automatically. You just call loss.backward().

Backpropagation: Chain Rule in Action

`L = (a + b) × c  with a=2, b=3, c=6  →  L=30`



Key sigmoid derivative: $\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$ — this is why sigmoid causes vanishing gradients (max value is just 0.25)

Loss Functions: Pick the Right One

Task	Loss Function	Output Layer	PyTorch
Regression	MSE / MAE	Linear (no activation)	`nn.MSELoss()`
Binary Classification	Binary Cross-Entropy	Raw logits (sigmoid inside loss)	`nn.BCEWithLogitsLoss()`
Multi-class	Cross-Entropy	Raw logits (softmax inside loss)	`nn.CrossEntropyLoss()`

Common trap: Don't apply sigmoid/softmax in your model's forward() AND use these loss functions. PyTorch's loss functions handle the activation internally — double-applying breaks training.

Today's Coding Session

~8 min

PyTorch Warm-up

Tensors, shapes, dtypes, matrix multiply — getting comfortable with the syntax

~15 min

XOR Problem

Why linear models fail, build a 2-2-1 network, watch it learn non-linearity

~15 min

Fashion-MNIST

End-to-end PyTorch workflow: data → model → train → evaluate on real images

+ 10 min Q&A at the end — bring your questions!

Questions?

Module 1: Introduction to ANNs

IIT Madras · Wadhwani School of AI